NFS Implementation Guide

Distributed GPU Workload Orchestrator — Shared Storage & User Access

1. Why NFS

The central server does not run the actual training jobs — worker machines do, and each worker's job needs access to the same training scripts, datasets, and log files as every other worker, without the system copying files around manually. A Network File System (NFS) solves this by letting one machine "export" a folder, and every other machine "mount" that same folder so it appears as an ordinary local folder — no custom file-transfer code required anywhere in the project.

Once mounted, our application code never needs to know NFS exists. A worker agent can simply `open()` a file at `/mnt/shared_jobs/...` exactly as if it were local — the OS and the NFS protocol handle the network transfer invisibly underneath.

2. Roles: server vs. client

- **NFS server** — one machine that owns the shared folder and exports it. In this project, the same machine running the central FastAPI server is a natural choice.
- **NFS clients** — every other machine that needs access, including all worker machines, and any student/professor machine that will place scripts directly (per our direct-access design).

3. Step 1 — Set up the NFS server

Install the NFS server package:

```
sudo apt update
sudo apt install nfs-kernel-server
```

Create the folder that will be shared:

```
sudo mkdir -p /mnt/shared_jobs
sudo chown nobody:nogroup /mnt/shared_jobs
sudo chmod 777 /mnt/shared_jobs
```

Note: this initial `777` permission is only a starting point to confirm the share works. Section 7 replaces this with a proper per-user permission model.

Tell NFS which folder to share and who is allowed to access it, by editing `/etc/exports`:

```
sudo nano /etc/exports

# Add this line (replace with your lab's actual IP range):
/mnt/shared_jobs 192.168.1.0/24(rw,sync,no_subtree_check)
```

Apply the change and restart the service:

```
sudo exportfs -ra
sudo systemctl restart nfs-kernel-server
```

4. Step 2 — Set up each client (workers and user machines)

Install the NFS client package and create the mount point:

```
sudo apt update
sudo apt install nfs-common
sudo mkdir -p /mnt/shared_jobs
```

Mount the server's shared folder (replace `192.168.1.10` with the actual server IP):

```
sudo mount 192.168.1.10:/mnt/shared_jobs /mnt/shared_jobs
```

5. Step 3 — Make the mount permanent

Without this step, the mount disappears on reboot and must be re-run manually every time. Add a line to `/etc/fstab` on every client machine:

```
192.168.1.10:/mnt/shared_jobs /mnt/shared_jobs nfs defaults 0 0
```

6. Step 4 — Test the shared folder

From the server:

```
echo "hello from server" > /mnt/shared_jobs/test.txt
```

From any client machine:

```
cat /mnt/shared_jobs/test.txt
```

If the client can read what the server wrote, the shared folder is working — any file placed there by any machine is instantly visible to every other machine.

7. Access control: giving every user their own isolated space

Standard NFS identifies a user by numeric UID (User ID), not by username or password — when a client writes a file, it simply tells the server "this file belongs to UID 1005", and the server trusts that claim. This means real isolation between users depends on Linux account setup and folder permissions, not on anything our application code does.

7.1 Create a matching Linux account for every user, on every machine

The critical detail: the same username must have the **same UID number** on the server and on every client it will touch. Mismatched UIDs across machines is the single most common real-world NFS mistake.

```
sudo useradd -u 1005 alice
# repeat with the same UID (1005) for 'alice' on every machine
```

7.2 Create a group for worker service accounts

Worker agents run as their own service account, not as any individual student or professor. To let workers read a user's script without giving them full access to everything, create a shared group and add every worker's service account to it:

```
sudo groupadd compute_workers
sudo usermod -aG compute_workers worker_service_account
```

7.3 Create each user's private folder

```
sudo mkdir -p /mnt/shared_jobs/users/alice
sudo chown alice:compute_workers /mnt/shared_jobs/users/alice
```

7.4 Set the permission level

Rather than locking the folder to the owner only (which would block workers from reading the script to run it), the folder is set so the owner has full read/write access, the compute_workers group can read only, and everyone else is blocked entirely:

```
sudo chmod 750 /mnt/shared_jobs/users/alice
```

750 = owner: read + write + enter; group (compute_workers): read + enter only; others: no access at all. This is what lets a worker machine read Alice's script to execute it, while Bob still cannot see or enter Alice's folder.

8. Recommended folder structure

```
/mnt/shared_jobs/
  users/
    alice/
      myenv/ (Alice's own Python virtual environment)
      job_17/
        train.py
        log.txt
    bob/
      ...
```

9. Runtime flow, once setup is complete

- **User side:** the user mounts the shared folder from their own machine, creates their own Python virtual environment inside their folder, and places their training script there.
- **Submission:** the user submits a job through the dashboard, giving only the script's relative path (e.g. `users/alice/job_17/train.py`) — the file itself is never uploaded, since it already exists on shared storage.
- **Scheduling:** the central scheduler assigns the job to a worker and a specific GPU, exactly as in the scheduling design covered separately.
- **Execution:** the worker agent — running under the compute_workers group — reads the script from the user's folder (allowed by the 750 permission), sets `CUDA_VISIBLE_DEVICES` to target the assigned GPU, and launches it as a subprocess.
- **Logging:** output is written to a log file inside the same user folder, so no separate file-transfer step is needed.
- **Viewing results:** the dashboard reads the log file directly, since the owning user's own account always has full access to their own folder.

10. Honest scope boundary

> *Plain NFS (v3, or v4 without Kerberos) trusts whatever UID a client claims — it does not cryptographically verify identity. This means a user with administrator/root access on their own machine could, in principle, impersonate another UID. True cryptographic authentication would require NFSv4 with Kerberos (a Kerberos KDC server, keytabs, realm configuration) — genuinely heavy infrastructure that is out of scope for this project. File-level isolation here is enforced via standard Unix/NFS permissions, which is an appropriate and realistic trust model for a managed lab network of known users — the same level of trust many real university HPC clusters operate under — but it is not a substitute for Kerberized NFSv4 in a fully untrusted, multi-tenant environment.*

11. Quick reference — command summary

Task	Command
Install NFS server	`sudo apt install nfs-kernel-server`
Export a folder	`edit /etc/exports, then: sudo exportfs -ra`

Install NFS client	`sudo apt install nfs-common`
Mount manually	`sudo mount <server-ip>:/mnt/shared_jobs /mnt/shared_jobs`
Mount permanently	`add a line to /etc/fstab`
Create matching user	`sudo useradd -u <uid> <username>`
Create worker group	`sudo groupadd compute_workers`
Set folder ownership	`sudo chown <user>:compute_workers <folder>`
Set folder permission	`sudo chmod 750 <folder>`